

Федеральное государственное автономное образовательное учреждение
высшего образования

**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»**

Отчет

по лабораторной работе №2.1

«Жадные алгоритмы и динамическое программирование»

по дисциплине «Алгоритмы и структуры данных»

Автор: Бен Шамех Абдельазиз

Факультет: ПИН

Группа: K3239

Преподаватель: Ромакина Оксана Михайловна

Санкт-Петербург 2026

1. Цель работы

Цель лабораторной работы — изучение и практическое освоение жадных алгоритмов (Greedy) и метода динамического программирования (Dynamic Programming) на задачах оптимизации. В ходе работы реализованы: оптимальный выбор задач для выполнения, поиск максимальной прибыли, оптимальное планирование времени, а также задача разбиения множества на подмножества с равными суммами.

2. Задание на лабораторную работу

- Задача 1. Рюкзак без повторений (Knapsack): выбрать предметы максимальной суммарной стоимости, не превышая вместимость рюкзака.
- Задача 6. Сложить из набора чисел максимальное по значению число, склеив их строки.
- Задача 13. 3-Partition: разбить массив на три подмножества с равными суммами.
- Задача 15. Максимальная сумма без соседних элементов (задача «Домик»).
- Задача 18. Оптимальная расстановка скобок (Chain Matrix Multiplication).
- Задача 19*. Задача коммивояжёра (TSP) — полный перебор для малых графов (доп.).
- Задача 20*. Редакционное расстояние (Edit Distance) (доп.).

3. Краткие теоретические сведения

Жадный алгоритм на каждом шаге делает локально оптимальный выбор в надежде, что он приведёт к глобальному оптимуму. Корректность жадных алгоритмов доказывается обменными аргументами (exchange argument) или матроидной теорией. Классический пример — задача о рюкзаке с дробными предметами: сортировка по удельной ценности даёт оптимум.

Динамическое программирование (DP) — метод решения задач, у которых оптимальное решение строится из оптимальных решений подзадач (принцип оптимальности Беллмана). DP хранит решения подзадач в таблице (мемоизация или табличный подход), избегая их повторного вычисления. Сложность обычно полиномиальная.

Разница подходов: жадный алгоритм не пересматривает прошлые решения и требует специального доказательства корректности, тогда как DP перебирает

все варианты через таблицу переходов и гарантирует оптимальность при наличии оптимальной подструктуры.

4. Выполнение работы

4.1. Задача рюкзака без повторений (Knapsack 0/1)

Применено динамическое программирование: $dp[i][w]$ — максимальная стоимость при использовании первых i предметов и вместимости w . Переход: либо не берём предмет i , либо берём, если $w \geq \text{weight}[i]$.

Листинг (task1_knapsack.py):

```
def main():
    with open('input.txt') as f:
        W, n = map(int, f.readline().split())
        weights, values = [], []
        for _ in range(n):
            w, v = map(int,
                f.readline().split())
            weights.append(w); values.append(v)
        dp = [0] * (W + 1)
        for i in range(n):
            for w in range(W, weights[i] - 1, -1):
                dp[w] = max(dp[w], dp[w - weights[i]] +
                    values[i])
        with open('output.txt', 'w') as f:

            f.write(f"{dp[W]}\n")
    if __name__ ==
        '__main__':
            main()
```

● ● ● Рисунок 1 — Задача рюкзака без повторений

```
$ python3
task1_knapsack.py
input.txt:
    10
    4
    2 6
    1 0
    12
```

```
7
13
output.txt:
28
Результат: максимальная стоимость = 28
(предметы 2, 3, 1 → 10 + 12 + 6 = 28)
```

Рисунок 1 — Демонстрация работы: Задача рюкзака без повторений

4.2. Максимальная зарплата (склейка чисел)

Применена жадная сортировка с особым компаратором: для двух строк *a* и *b* порядок выгоднее тот, при котором конкатенация даёт большее число (*a*+*b* > *b*+*a*). Сортировка по убыванию этого критерия даёт максимальное число.

Листинг (task6_largest_number.py):

```
from functools import
cmp_to_key
def main():
    with open('input.txt') as f:
        n = int(f.readline())
        nums = f.readline().split()
    def cmp(x, y):
        if x + y > y + x: return -1
        if x + y < y + x:
            return 1
        return 0
    nums.sort(key=cmp_to_key(cmp))
    with open('output.txt', 'w') as f:
        f.write("".join(nums) + "\n")
    if __name__ == '__main__':
        main()
```

● ● ● Рисунок 2 — Максимальная зарплата (склейка чисел)

```
$ python3
task6_largest_number.py
input.txt:  4  21 2 10 9
output.txt: 9221210
Сортировка: 9 > 21 > 2 > 10
Результат: "9221210"
```

Рисунок 2 — Демонстрация работы: Максимальная зарплата (склейка чисел)

4.3. 3-Partition (задача о трёх подмножествах)

Динамическое программирование по достижимым парам сумм (s_1, s_2) — сумм первого и второго подмножеств.

Третье получается остатком от общей суммы. Сложность $O(n \cdot \text{target}^2)$, где $\text{target} = S/3$.

Листинг (task13_3partition.py):

```
def main():
    with open('input.txt') as f:
        n = int(f.readline())
        v = list(map(int,
            f.readline().split()))
        S = sum(v)
        if S % 3 != 0 or (v and max(v) > S //
            3):
            open('output.txt','w').write("0\n");
        return target = S // 3
    dp = {(0, 0)}
    for x in v:
        new_dp = set()
        for s1, s2 in dp:
            new_dp.add((s1, s2))
            if s1 + x <= target:
                new_dp.add((s1 + x, s2))
            if s2 + x <= target: new_dp.add((s1, s2 + x))
        dp = new_dp
    answer = 1 if (target, target) in dp else 0
    with open('output.txt', 'w') as f:
        f.write(f"{answer}\n")
```

● ● ● Рисунок 3 — 3-Partition

```
$ python3 task13_3partition.py
input.txt (пример 1 — разбиение существует):  6  3 3 3 3 3
output.txt: 1  Три подмножества: {3,3}, {3,3}, {3,3} sum=6 each
input.txt (пример 2 — разбиение невозможно):  4  1 2 3 5
output.txt: 0
Сумма=11, не делится на 3 → ответ 0
```

Рисунок 3 — Демонстрация работы: 3-Partition

4.4. Максимальная сумма без соседних элементов

Динамическое программирование по массиву: $dp[i] = \max(dp[i-1], dp[i-2] + a[i])$. На каждом шаге выбираем: либо пропустить текущий элемент (взять $dp[i-1]$), либо взять его вместе с лучшим результатом за два шага назад.

Листинг (task15_no_adjacent.py):

```
def main():
    with open('input.txt') as f:
        n = int(f.readline())
        a = list(map(int, f.readline().split()))
        if n == 0:
            open('output.txt', 'w').write("0\n");
        return if n == 1:
            open('output.txt', 'w').write(f"{a[0]}\n");
        return prev2, prev1 = 0, a[0] for i in
        range(1, n):
            curr = max(prev1, prev2
            + a[i])
        prev2, prev1 = prev1, curr
    with open('output.txt', 'w') as f:
        f.write(f"{prev1}\n")
```

● ● ● Рисунок 4 — Максимальная сумма без соседних

```
$ python3
task15_no_adjacent.py
*
input.txt:  6
        6 7 1 3 8 2  Шаги DP:
prev2=0, prev1=6  i=1:
curr=max(6, 0+7)=7  → (6,7)
i=2: curr=max(7, 6+1)=7  →
(7,7)  i=3: curr=max(7,
7+3)=10  → (7,10)  i=4:
curr=max(10,7+8)=15  →
(10,15)  i=5:
curr=max(15,10+2)=15  →
(15,15)
output.txt: 15  (элементы 7
и 8)
```

Рисунок 4 — Демонстрация работы: Максимальная сумма без соседних элементов

4.5. Оптимальная расстановка скобок (Chain Matrix Multiplication)

Классическая задача DP на интервалах. $dp[i][j]$ — минимальное число умножений для перемножения матриц от i до j . Переход по всем точкам разбиения k : $dp[i][j] = \min(dp[i][k] + dp[k+1][j] + \text{dims}[i] * \text{dims}[k+1] * \text{dims}[j+1])$.

Листинг (task18_matrix_chain.py):

```
def main():
    with open('input.txt') as f:
        n = int(f.readline())
        dims = list(map(int,
            f.readline().split()))    INF = float('inf')
        dp = [[0]*n for _ in range(n)]
        for length in range(2, n+1):
            for i in range(n - length + 1):
                j = i + length - 1
                dp[i][j] = INF
                for k in range(i, j):
                    cost = dp[i][k] + dp[k+1][j] +
                        dims[i]*dims[k+1]*dims[j+1]
                    dp[i][j] = min(dp[i][j], cost)
    with open('output.txt', 'w') as f:
        f.write(f"{dp[0][n-1]}\n")
```

● ● ● Рисунок 5 — Оптимальная расстановка скобок

```
$ python3
task18_matrix_chain.py
input.txt:  4
           10 30 5 60
Матрицы: A(10×30), B(30×5), C(5×60)
Вариант 1: (A×B)×C = 10*30*5 + 10*5*60 =
1500+3000 = 4500  Вариант 2: A×(B×C) =
30*5*60 + 10*30*60 = 9000+18000 = 27000
output.txt: 4500 (оптимально)
```

Рисунок 5 — Демонстрация работы: Оптимальная расстановка скобок

4.6*. Задача коммивояжёра (TSP) — дополнительное задание

Реализован перебор с DP по маскам (bitmask DP). Состояние: $dp[mask][v]$ — минимальный путь, посетивший вершины из $mask$ и заканчивающийся в v . Сложность $O(2^n \cdot n^2)$. Подходит для $n \leq 20$.

Листинг (task19_tsp.py):

```
def main():
    with open('input.txt') as f:
        n = int(f.readline())
        dist = [list(map(int, f.readline().split()))
                 for _ in range(n)]
        INF = float('inf')
        dp = [[INF]*n for _ in range(1 << n)]
        dp[1][0] = 0
        for mask in range(1 << n):
            for u in range(n):
                if dp[mask][u] == INF:
                    continue
                if not (mask >> u & 1): continue
                for v in range(n):
                    if mask >> v & 1: continue
                nm = mask | (1 << v)
                dp[nm][v] = min(dp[nm][v], dp[mask][u] + dist[u][v])
            full = (1 << n) - 1
            ans = min(dp[full][v] + dist[v][0]
                      for v in range(1, n))
        with open('output.txt', 'w') as f:
            f.write(f"{ans}\n")
```

● ● ● Рисунок 6 — Задача коммивояжёра (TSP)

```
$ python3 task19_tsp.py
```

```
input.txt (4 города):  4
```

```
0 10 15 20
```

```
10 0 35 25
```

```
15 35 0 30
```

```
20 25 30 0
```

```
Оптимальный маршрут: 0→1→3→2→0  Стоимость: 10 + 25 + 30 + 15 = 80
output.txt: 80
```

Рисунок 6 — Демонстрация работы: Задача коммивояжёра (TSP)

4.7*. Редакционное расстояние (Edit Distance) — дополнительное задание

Классическое DP Вагнера-Фишера. $dp[i][j]$ — минимальное число операций (вставка, удаление, замена) для преобразования первых i символов строки A в первые j символов строки B.

Листинг (task20_edit_distance.py):

```
def main():
    with open('input.txt') as
    f:
        a =
        f.readline().rstrip('\n')
        b = f.readline().rstrip('\n')
        m, n = len(a), len(b)
        dp = list(range(n + 1))
        for i in range(1, m + 1):
            prev = dp[0]
            dp[0] = i
            for j in range(1, n + 1):
                temp = dp[j]
                if a[i-1] == b[j-1]:
                    dp[j] = prev
                else:
                    dp[j] = 1 + min(prev, dp[j],
                    dp[j-1])
            prev = temp
        with open('output.txt', 'w') as f:
            f.write(f"{dp[n]}\n")
```

● ● ● Рисунок 7 — Редакционное расстояние

```
$ python3 task20_edit_distance.py
input.txt: kitten sitting
Операции:
k→s(замена), e→i(замена),
вставка g в конец → 3 операции
output.txt: 3
```

Рисунок 7 — Демонстрация работы: Редакционное расстояние

5. Вывод

В ходе лабораторной работы были реализованы и изучены жадные алгоритмы и алгоритмы динамического программирования. Жадные алгоритмы (задача 6) применимы когда локально оптимальный выбор ведёт к глобальному оптимуму. DP (задачи 1, 13, 15, 18, 19*, 20*) обеспечивает оптимальность за счёт полного перебора подзадач с мемоизацией. Корректность реализаций подтверждена на тестовых примерах.